

Today's Agenda:-

1. Intro To Strings
2. Some Questions on Strings.
3. Immutability of Strings.

String :- An array / sequence of characters.

Character :- Any symbol that is understandable by my machine.

ex :- 'A', 'B', 'c'

'a', 'b', '0', '1', '2', '\$'

'_', 'x'

Q1- How do computers understand character?
ASCII

" "

→ 32:

ASCII Values

'A' → 65	'a' → 97	'0' → 48
'B' → 66	'b' → 98	'1' → 49
⋮	⋮	'2' → 50
'Z' → 90	'z' → 122	'9' → 57

Some Operations

1. `char ch = (char) 65;`
 `print(ch);` 'A'

2. `char ch = (char) ('a' + 1);`
 `print(ch);` 'b'

3. `int x = 'a';`
 `print(x);` 97;

Switch Case

< **Question** > : Given a string consisting of lower-case and upper-case alphabets.

Convert: (1) lowercase $\rightarrow$ uppercase

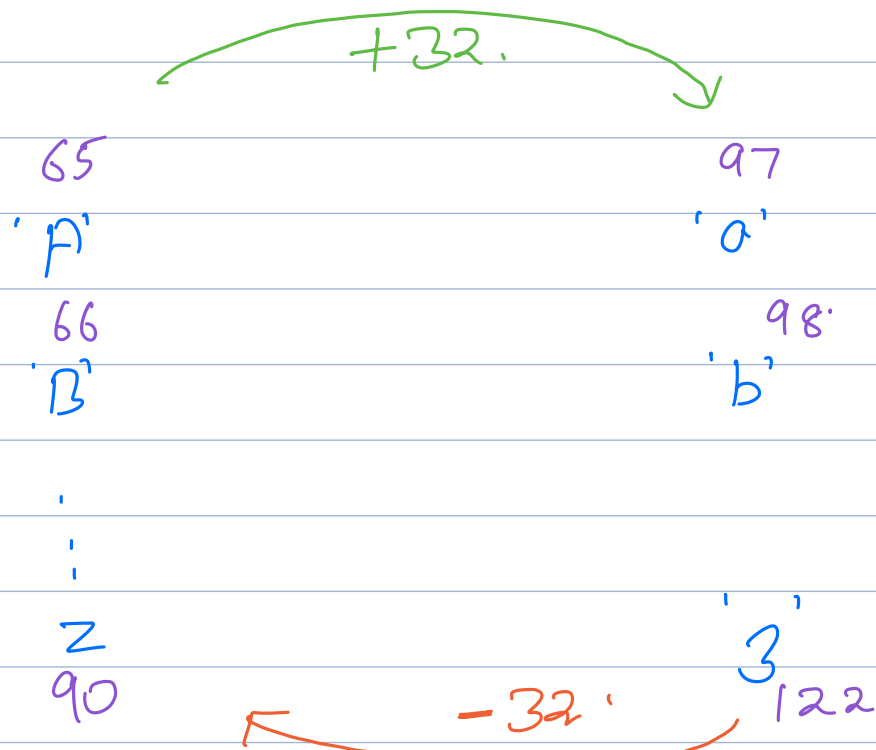
(2) uppercase $\rightarrow$ lowercase

1. "Hello" - "hELL0"

Q:- "aDgbHJe"

"AdGBhje"

Idea:-



Approach:- Iterate on every character of the input string.

If the char is uppercase.
 $65 \leq ch \leq 90$. add 32 to convert it to correspondingly lower case.
else. subtract 32. to convert to correspondingly upper case.

Code

C++;

String toggle (String s) {

int size = s.size();

for (i=0; i < size; i++) {

if (s[i] >= 65 && s[i] <= 90) {
s[i] = (char) (s[i] + 32);

} else {

s[i] = (char) (s[i] - 32);

}

return s;

}

T.C $\rightarrow O(N)$

S.C $\rightarrow O(1)$

Java / Python → String are immutable.

String toggle (String s) {

int size = s.size(); String ans = "";

for (i=0; i < size; i++) {

if (s[i] > 65 && s[i] ≤ 90) {
ans = ans + (char) (s[i] + 32);

} else {

ans = ans + (char) (s[i] - 32);

}

}

return ans;

"aBcD"
i

ans = ""

↓

"A"

4k

"AB"

12k.

ans = "ABC"

3

"abcde"

1

(A)

2

AB

3

ABC

4

ABCD

5

ABCDE

For a string of size N .

$1 + 2 + 3 + 4 \dots N$ iterations.

$$\frac{N(N+1)}{2}$$

$$\underline{T.C \rightarrow O(N^2)}$$

1. Using String Builder

Amortised $O(1)$

Avg $O(1)$

```
StringBuilder sb = new StringBuilder();  
for (int i = 0; i < n; i++) {  
    sb.append('a'); // Adds a character efficiently  
}  
String result = sb.toString(); // Converts StringBuilder to String
```

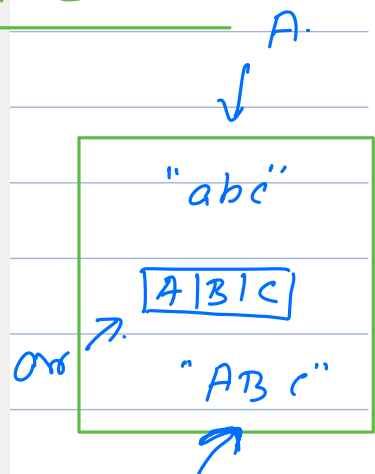
$S.C \rightarrow O(N)$

2. To Char Array

```
----JAVA----  
String solve(String A) {  
    char arr[] = A.toCharArray(); //for java  
  
    for(int i=0; i < arr.length; i++) {  
        if(arr[i] >= 'A' && arr[i] <= 'Z') {  
            arr[i] = (char)(arr[i] + 32);  
        }  
        else {  
            arr[i] = (char)(arr[i] - 32);  
        }  
    }  
  
    return new String(arr);  
}
```

```
----PYTHON----  
class Solution:  
    def solve(self, A):  
        char_list = []  
        for c in A:  
            if 'A' <= c <= 'Z':  
                char_list.append(chr(ord(c) + 32))  
            else:  
                char_list.append(chr(ord(c) - 32))  
        return ''.join(char_list)
```

$S.C \rightarrow O(N)$



Q1

"abcdef"

"a"

ab

abc

abcd

abcde

abcdef

Substring

1. Contiguous part of a string.
2. A single character is also a substring.
3. Whole string is also a substring.
4. Empty string (" ") is not a substring.
5. String of length N. How many substrings will be there?

$$\frac{N(N+1)}{2}$$

ex:- $S = "a^0 b^1 c^2"$

Substrings

a b c
a b b c
a b c

Ans:- $S = "b^0 x^1 c^2 d^3"$

ans = 10

b x c d
b x x c c d
b x c x c d
b x c d

Check for substring if it's a palindrome or not.

"NAYAN"

"mom"

"P A C E C A R"

"MALAYALAM"

tone:

ex:- $S = \text{"ana madam spe"}$

na x an → false:

```
bool isPalindrome (String str, int s, int e) {
```

```
while ( s ≤ e ) {
```

```
if (sto[s] != sto[e]) {  
    return false;  
}
```

3 $g^{++}; e^{--};$

3

```
return true;
```

3

$T.C \rightarrow O(N)$

$$S-C \rightarrow O(1)$$

< Question > : Given a string s. Find the length of the longest palindrome substring in s.

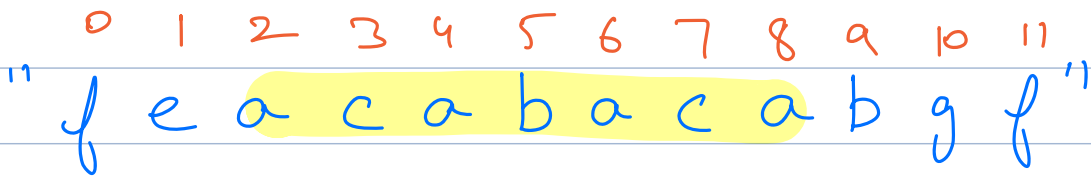
ex:- "a n a m a d a m n"



ans = 5.

Q:-

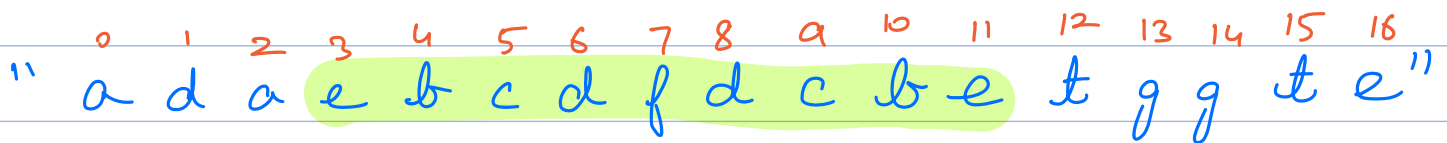
" f e a c a b a c a b g f "



7.

Q:-

" a d a e b c d f d c b e t g g t e "



9.

Idea 1: Consider every substring & check if it is palindromic or not. If yes, compare the ans with the length of this S-S & update if needed.

```
int IPS (String str) {
```

```
    int ans = 1;
```

```
    for (s=0; s < str.length(); s++) {
```

```
        for (l=s; l < str.length(); l++) {
```

```
            bool isPalin = isPalin(str, s, l);
```

```
            if (isPalin == true) {
```

```
                ans = max(ans, l-s+1);
```

```
            }
```

```
        }
```

```
    return ans;
```

```
}
```

T.C $\rightarrow O(N^3)$

S.C $\rightarrow O(1)$.

8:30

Idea 2: Since every palindromic string is symmetric around its center. Consider every index as the center & expand it in both directions. Because a palindromic can be of even length (2 centers) &

of odd length (1 center); while iterating
on every idx, I need to expand
in both dir. by considering

a) i as centre

b) i & i+1 as centre.

c

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
" a d a e b c d f d c b e t g g t e "

1 8

ans = 1, 9.

$l = 2$

$r = 12$

(1, 8)

(2, 12)

$$12 - 2 - 1 = 9$$

```
int LPS (String s) {
```

```
    int ans = 1;    int N = s.size();
```

```
    for (c = 0; c < N; c++) {
```

c as centre.

```
        l = c-1; r = c+1;
```

```
        while ( l >= 0 && r < N &&
```

```
            s[l] == s[r] ) {
```

l--; r++;

3

ans = max(ans, r - l - 1);

C & C+1 as center.

l = C; r = C + 1;

while (l >= 0 && r < N &&
s[l] == s[r]) {

l--; r++;

3

ans = max(ans, r - l - 1);

3

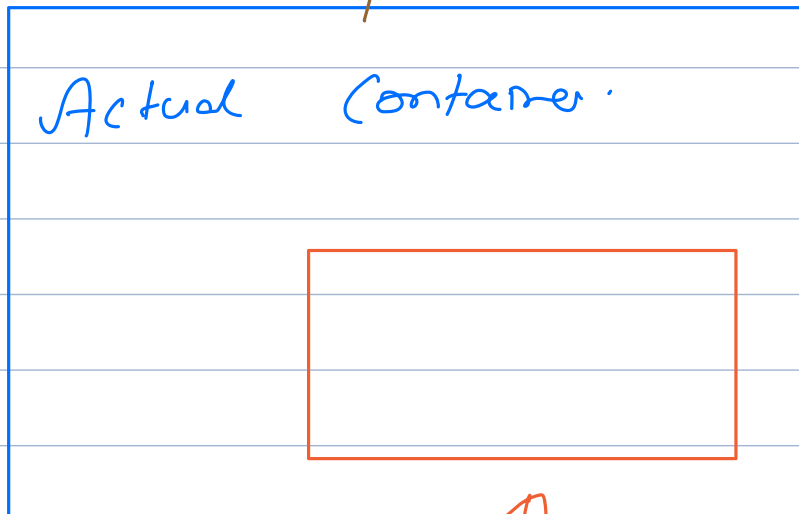
return ans;

T.C $\rightarrow O(N^2)$.

S.C $\rightarrow O(1)$.

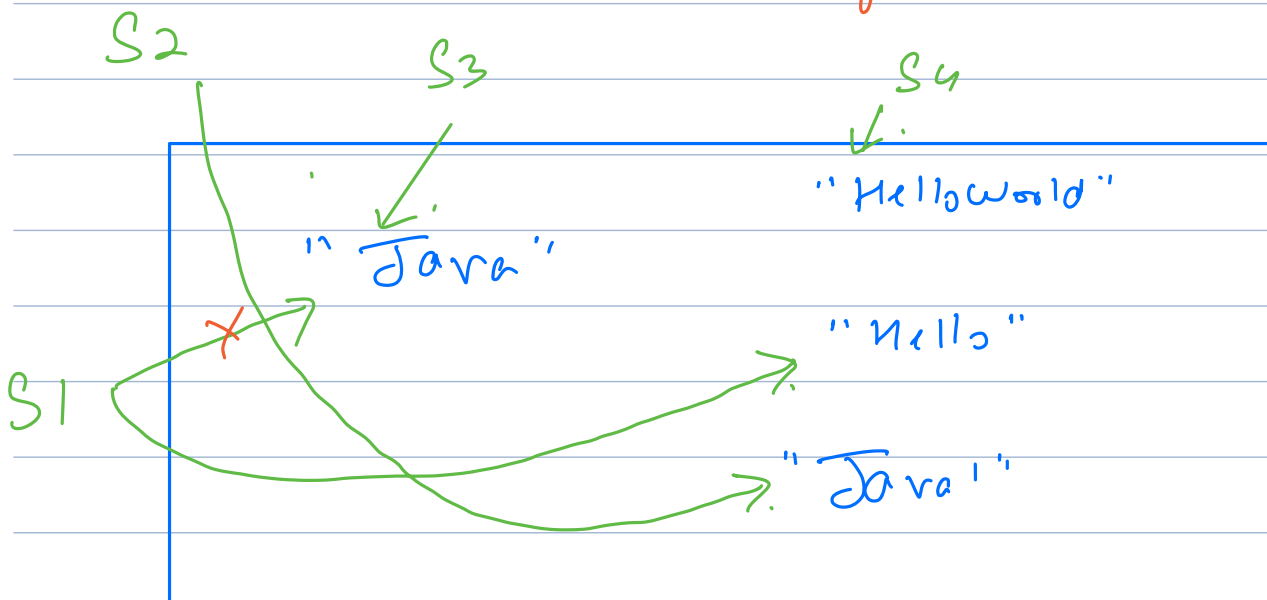
Immutability of Strings

Heap.



Active String Pool

String Constant Pool.



Active String Pool.

1. String s1 = "Java";

2. String s2 = "Java";

3. s1 = "Hello";

4. String s3 = s2;

5. s1.concat("World");

6. String s4 = s1.concat("World");

7. String s2 = "Java";

